Take-Home Assignment

Hybrid Rendering Admin Dashboard (Next.js + Node.js)

Field	Details
Role	Full Stack Developer (0–1 years experience)
Stack	Next.js (App Router), Node.js / Express, MongoDB
Estimated effort	10–16 hours (spread over 5–7 days)
Submission	GitHub repo link + deployed link + short Loom/video walkthrough (8–12 min)
AI tools	Allowed and encouraged — see 'Use of AI Tools' section below

1. Overview

You will build a small admin dashboard for managing a list of items (e.g., products for an e-commerce store, or posts for a blog — pick whichever you prefer, the data model is the same shape). The core idea of this assignment is to test whether you understand how rendering works in Next.js, not just whether you can build a CRUD UI.

The dashboard has two halves that behave very differently:

- **The initial item list** must be rendered on the server (Server Components / SSR) so that it is present in the HTML on first load — useful for SEO and fast first paint.
- **Filtering, searching, and pagination** must happen on the client without a full page reload — the URL should still update (so results are shareable/bookmarkable), but the page should not do a hard refresh.

You are free (and encouraged) to use AI tools like Claude or ChatGPT while building this. The assignment is designed so that simply pasting this brief into an AI tool and submitting the output will not be enough — see the 'Use of AI Tools' and 'Evaluation' sections for why.

2. Scenario

You're building the internal admin tool for a small online store. The store's catalog team needs a page where they can:

1. View a list of all products, paginated (server-rendered on first load).
2. Search products by name (client-side, no full reload).
3. Filter products by category and by stock status (in stock / out of stock) (client-side, no full reload).
4. Click a product to view/edit its details on a separate page.
5. Add a new product via a form.
6. Delete a product with a confirmation step.

The catalog team cares about page load speed for the main list (it's checked by an SEO tool occasionally) but doesn't want the page to reload every time they type in the search box or change a filter.

3. Functional Requirements

3.1 Backend (Node.js / Express + MongoDB)

- Build a REST API with the following endpoints (or equivalent):
  - GET /api/products — supports query params: page, limit, search, category, status (in-stock/out-of-stock)
  - GET /api/products/:id — get a single product
  - POST /api/products — create a product
  - PUT /api/products/:id — update a product
  - DELETE /api/products/:id — delete a product
- Each product should have at minimum: name, description, price, category, stock (number), createdAt
- Seed the database with at least 60 products spanning at least 4 categories, with a mix of in-stock and out-of-stock items (a seed script should be included).
- Pagination should be done at the database level (skip/limit or cursor-based) — not by fetching everything and slicing in JS.

3.2 Frontend (Next.js — App Router preferred)

Product List Page (/products)

- **Initial render:** the first page of products (with default filters) must be fetched and rendered on the server, so that 'View Page Source' or a disabled-JS request shows the product names in the HTML.
- **Search box:** typing in the search box filters the list without a full page reload. Debounce the input (e.g., 300–500ms) so you're not firing a request on every keystroke.
- **Category & status filters:** dropdowns/checkboxes that update the list without a full reload.
- **Pagination:** next/previous (or numbered) controls that update the list without a full reload.
- **URL sync:** search, filters, and page number must be reflected in the URL query string (e.g., /products?search=shoe&category=footwear&status=in-stock&page=2), so a user can refresh or share the URL and get the same view.
- **Loading state:** show a visible loading indicator while client-side requests are in flight (skeleton, spinner, etc.) — the old list shouldn't just freeze with no feedback.

Product Detail / Edit Page (/products/[id])

- Shows product details and an editable form.
- Saving updates the product via the API and reflects the change when the user navigates back to the list (no stale data).

Add Product Page (/products/new)

- A form to create a new product, with both client-side and server-side validation (required fields, price ≥ 0, stock ≥ 0).

Delete

- Deleting a product requires a confirmation step (modal or confirm dialog) and removes it from the list view without a full page reload.

4. Specific Constraints (Read Carefully)

These constraints are the actual point of the assignment

• The product list page must NOT be a fully 'use client' page that fetches everything on mount via useEffect. The first render must come from the server.

• Filtering/search/pagination must NOT cause a full document reload (no plain <a href> navigation or window.location.href for these — but the URL must still change).

• You must be able to explain, for every part of the page, why it is a Server Component or a Client Component.

It's fine — and often correct — to have a hybrid structure, e.g., a Server Component that fetches the initial data and renders the static shell, with a nested Client Component that owns the search/filter/pagination state and re-fetches via an API route or fetch call as needed. There isn't only one correct architecture, but your choice needs to make sense and you need to be able to defend it.

5. Core Requirements — Advanced (Problem Solving) (Implement 2-3 features out of all these)

The requirements below are part of the core assignment, not bonus work. They're intentionally not things a generic AI-generated CRUD app handles by default — you need to recognize that these are problems before you can ask an AI tool (or yourself) to solve them. We've deliberately kept the list short so you have time to get each one right rather than rushing through many features.

5.1 Out-of-order search responses (race condition)

If a user types quickly in the search box, multiple requests can be in flight at once. A slower earlier request can resolve after a faster later one and overwrite the screen with stale results.

- The UI must always reflect the result of the most recently issued request, regardless of response order.
- Use an approach such as AbortController, request sequence numbers, or an equivalent — and be ready to explain why it works.

5.2 Optimistic UI with rollback (stock toggle)

Add a quick 'Mark as Out of Stock / In Stock' toggle directly in the product list (no need to open the edit page).

- Clicking the toggle must update the UI immediately (optimistic update), before the server responds.
- If the API call fails (simulate this — e.g., a query param like ?simulateError=true that your backend can use to force a failure for testing), the UI must roll back to the previous state and show an error message.

- Be able to explain the difference between your optimistic client state and the server's source-of-truth state.

5.3 Live category counts via aggregation

Next to each category in the filter UI, show a live count of matching products, e.g., 'Footwear (12)'.

- Counts must be computed server-side using a MongoDB aggregation pipeline ($match / $group / $facet or similar) — not by fetching all products and counting in JavaScript.
- Counts must respect the other currently active filters (search term and stock status) — e.g., if the user has searched for 'shoe' and filtered to in-stock only, the category counts shown should reflect that subset, not the whole catalog.

5.4 Cache & revalidation correctness

The product list's first load is server-rendered (per Section 4). After a product is added, edited, or deleted:

- The server-rendered list must show the updated data on the next full load/navigation (not cached stale data).
- The currently-open client-side filtered/paginated view must also reflect the change (e.g., after editing a product's stock status from its detail page and returning to the list, it should show the new status — without a manual hard refresh).
- *Solving this by disabling all caching everywhere (e.g., `force-dynamic` on every route, no-store on every fetch) is not acceptable — it defeats the SSR requirement in Section 4 and will be treated as not meeting this requirement.*

5.5 Conflict detection on concurrent edits (optimistic locking)

Add a version field (a number, or reuse updatedAt) to the product schema.

- When saving an edit, the request must include the version/timestamp the form was loaded with.
- If the product has been changed by someone else since the form was loaded (i.e., the stored version no longer matches), the API must reject the update with a clear conflict response (e.g., HTTP 409) instead of silently overwriting.
- The UI must show a clear message when this happens (e.g., 'This product was updated elsewhere — please reload to see the latest version'), rather than failing silently or showing a generic error.
- To test this in your walkthrough: open the same product's edit page in two tabs, save in one, then save in the other, and show the conflict being caught.

6. Deliverables

7. A public GitHub repository containing the frontend (Next.js) and backend (Node/Express) code, with a clear README explaining how to run both locally, including environment variables needed.
8. A working deployed version (e.g., frontend on Vercel, backend on Render/Railway, MongoDB on Atlas). Include the live URL in the README.
9. A short (8–12 minute) screen-recorded walkthrough (Loom or similar, link in README) covering:

- A quick demo of the working app (search, filter, pagination, add/edit/delete).
- Which parts of the product list page are Server Components vs Client Components, and why.
- How the URL stays in sync with filters/search/pagination without a full reload.
- A demo of each Section 5 item: race-condition handling, optimistic toggle + rollback, live category counts, cache/revalidation behavior after an edit, and the two-tab conflict scenario.
- One thing you'd improve or do differently with more time.

7. Use of AI Tools

You're welcome to use AI tools (Claude, ChatGPT, Copilot, etc.) at any stage — for scaffolding, debugging, or learning concepts you're unfamiliar with. This is normal in real development work and we're not trying to catch you out for using them.

That said:

- You are responsible for every line of code you submit. If we ask you to change something live in the interview, you should be able to do it.
- AI-generated solutions to this exact brief commonly get the Server/Client Component split wrong on the first attempt (e.g., putting everything in a Client Component, or fetching data twice — once on the server and again on the client). Submissions with this issue will be asked to explain and fix it live, so it's worth testing this yourself rather than assuming the AI got it right.
- Please don't remove comments or explanations that would help us understand your reasoning — if AI helped you write a tricky part, a short comment on why it works is appreciated.
- **The Section 5 requirements (race conditions, optimistic UI, aggregation counts, caching, conflict detection) are the parts most likely to separate candidates.** If you're not sure these apply to your implementation, that's a sign worth investigating before you submit — an AI tool won't add these on its own unless you specifically ask it to, and asking for the right thing requires recognizing the problem first.

8. Bonus (Optional, Not Required)

- Add loading.tsx / Suspense boundaries for a smoother loading experience.
- Use revalidation (e.g., revalidatePath or tags) so the list reflects newly added/edited products without a manual refresh.
- Add basic sorting (e.g., by price or stock).
- Add simple authentication so only logged-in users can add/edit/delete products.
- Write 2–3 basic tests (API route or component).

9. Submission Checklist

Before you submit, make sure you have:

☐ GitHub repo link (frontend + backend, with README and setup instructions)
☐ Live deployed link (frontend connected to live backend/database)

☐ Loom/video walkthrough link (5–8 minutes)
☐ Confirmed the deployed app actually works end-to-end (not just locally)

Good luck — we're looking forward to seeing your approach. If anything in this brief is ambiguous, make a reasonable assumption, note it in your README, and move on.